

Diverging Expressions

Document #: P3549R0 [[Latest](#)] [[Status](#)]
Date: 2025-01-12
Project: Programming Language C++
Audience: EWG
Reply-to: Bruno Cardoso Lopes
<bruno.cardoso@gmail.com>
Zach Laine
<whatwasthataddress@gmail.com>
Michael Park
<mcypark@gmail.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[2 Diverging Expressions](#)

[2.1 Diverging `do` expressions](#)

[2.2 A bottom type](#)

[2.2.1 `std::function<std::noreturn\_t\(\)>`](#)

[2.2.2 `std::expected<T, std::noreturn\_t>`](#)

[2.2.3 Valueless Ranges](#)

[2.3 Doing Our Due Diligence to Deduce `do` Divergence](#)

[2.4 Alternative with `\[\[noreturn\]\]`](#)

[3 Proposal](#)

[4 References](#)

§ 1 Introduction

One pattern that will occur with some regularity with pattern matching [[P2688R4](#)] ([Pattern Matching: match Expression](#)) (and [[P2806R2](#)] ([do expressions](#))) is the desire to produce values for some patterns but not for all cases. Consider the following simplified example:

```
void f(int i) {  
    int j = i match {  
        0 => 0;  
        _ => std::terminate();  
    };  
    use(j);  
}
```

```
void g(int i) {  
    int j = i match {  
        0 => 0;  
        _ => do {  
            std::terminate();  
        };  
    };  
}
```

```
void h(int i) {  
    int j = i match {  
        0 => 0;  
        _ => do -> int {  
            std::terminate();  
        };  
    };  
}
```

	<code>use(j);</code> <code>}</code>	<code>use(j);</code> <code>}</code>
--	--	--

In all of these cases, the desire is that if `i == 0` then we initialize `j` to the value `0`, otherwise we `std::terminate()`. Given that `std::terminate` does in fact, as the name suggests, terminate, we don't actually have to worry about producing a value in that case. These examples are all equivalent — or at least, they should be. But C++ does not currently recognize them as such (with one exception that we'll get to later).

The rule for pattern matching by default is that the type of the match expression is deduced from each arm and each arm has to have the same type. The same rule we have for `auto` deduction in functions and lambdas. In the above examples, the first arm always has type `int`. But for `f()` and `g()`, the second arm has type `void`. In order to type check, we have to wrap our call to `std::terminate` in an expression that actually has type `int`. The `do` expression in `h()` is one such way.

The extra `-> int` does solve the problem, but it's also misleading. We're not actually ever producing an `int` here, we're just writing a type annotation to hammer the compiler into submission. That's just not a great place to be.

Our goal with this paper is to have all of `f()`, `g()`, and `h()` above type check. Concretely: we need to recognize that an expression can *diverge* and then have pattern matching's type deduction rules only consider those arms that do not diverge. Importantly, we want to treat the expressions:

```
std::terminate()
```

and

```
do { std::terminate(); }
```

and

```
do { log::error("hasta la vista"); std::terminate(); }
```

as all being diverging expressions in order to facilitate code evolution. It would be annoying if a naked call to `std::terminate` worked but once you wrap it in a `do` expression that does so much as add a single log statement that it suddenly doesn't. We want to properly recognize divergence, not just the most trivial of cases.

§ 2 Diverging Expressions

An expression is said to *diverge* if it unconditionally escapes control flow. Currently in C++, we have two such expressions in the language:

- a *throw-expression*, and
- a call to a `[[noreturn]]` function.

The former explicitly diverges (control flow immediately escapes and doesn't return to keep evaluating in the original slot) while the latter implicitly diverges (technically a function marked `[[noreturn]]` can still return, we just state that doing so is undefined behavior in 9.12.11 [\[dcl.attr.noreturn\]](#)).

As hinted at earlier, the language already recognizes that expressions can diverge in one spot: the conditional operator. Consider:

```
int x = condition ? 42 : throw std::runtime_error("oops"); // OK
```

The conditional operator has to merge two values into one. But if one of the operands is a *throw-expression*, then the value of the conditional operator is trivially the other operand — or there is just no value to speak of. The above is perfectly valid code, the conditional expression is a prvalue `int` (that sometimes throws).

The pattern matching paper recognizes this as well, and explicitly recognizes pattern arms that are *throw-expressions* as not participating in deduction either:

```
int x = v match { // OK
    0 => 0;
    _ => throw std::runtime_error("oops");
};
```

But this does not work today, despite being just as diverging as a `throw`:

```
int x = condition ? 42 : std::terminate();
```

Instead you could do something like this (which we think is the kind of thing that really belongs on a T-shirt):

```
int x = condition ? 42 : throw (std::terminate(), 0);
```

Note that this *does* work — we will terminate before we evaluate the `throw`.

But there's not any difference between throwing an exception and invoking a non-returning function when we're talking about whether an expression produces a value. Neither produces a value, so neither needs to participate in any language rules that involve merging multiple expressions!

We think it's important to recognize invocations of non-returning functions as diverging because we want people to be able to write the code on the left, and not have to write either of the two workarounds presented:

Desired	Workaround with <code>throw</code>	Workaround with explicit type
<pre>void f(int i) { int j = i match { 0 => 0; _ => std::terminate(); }; use(j); }</pre>	<pre>void g(int i) { int j = i match { 0 => 0; _ => throw (std::terminate(), 0); }; use(j); }</pre>	<pre>void h(int i) { int j = i match { 0 => 0; _ => do -> int { std::terminate(); }; }; use(j); }</pre>

§ 2.1 Diverging `do` expressions

Consider the following expressions:

1. `throw 42`
2. `std::terminate()`
3. `do { throw 42; }`
4. `do { std::terminate(); }`

All four of these expressions diverge, the bottom two just trivially wrap the top two. So we'd want these to have the same properties when it comes to divergence — otherwise any slight refactoring could have too much impact. For instance, the desire to change:

```
std::terminate()
```

into

```
do {
  log::fatal("oops, I did it again");
  std::terminate();
}
```

should ideally not change any properties of the expression. Both diverge, one just additionally adds some logging.

One question we have to answer is: What is the `decltype` of such an expression?

Right now, `decltype(throw 42)` is defined as `void`, [explicitly](#). For some reason. `decltype(std::terminate())` is more obviously `void` simply because that's how the function is defined — it is a `void()` (although a `[[noreturn]]` one). So there's certainly something to be said for going ahead and wanting to define `decltype(do { throw 42; })` as `void` as well.

This approach leads to the following pair of rules:

1. the *type* of a `do` expression is either
 - the *trailing-return-type*, if one is explicitly provided, or
 - the type that is `do_return`-ed consistently (same as the lambda rule except with `do_return` instead of `return`).
2. A `do` expression is said to *diverge* if every control flow path leads to executing a diverging expression (i.e. a *throw-expression*, a call to a `[[noreturn]]` function, or evaluating another such diverging expression).

This effectively means that we keep adding properties to expressions: an expression would have a type, a value category, whether it's a bit-field, and now also whether it diverges.

There is another approach though.

§ 2.2 A bottom type

Several languages have a notion of a bottom type ($\perp$) for diverging expressions. This type is spelled `Nothing` in Scala, `never` in TypeScript, `Never` in Python, `noreturn` in D, `!` in Rust, `void` in Haskell (but notably not `void` in C++), etc. This type represents an expression that diverges.

Let's imagine what it would look like if C++ also had such a type. We'll call it `noreturn_t`. It would have some interesting properties:

- You cannot produce an instance of such a type. It wouldn't be very divergent otherwise! In this case, it's not just that this type isn't constructible, but we would also have to eliminate all of C++'s other clever ways of producing a value (like `reinterpret_cast`, `std::bit_cast`, etc.).
- On the other hand, `noreturn_t` is convertible to any other type (including reference types). This is currently impossible to emulate in C++ today, since you cannot make a type that is both convertible to `T` and `T&` for all `T`.
- Function pointer conversions are more interesting. From a type theory perspective, `auto(*) (T) -> noreturn_t` is convertible to `auto(*) (T) -> U` for all `U`. For the same reason Haskell has the [absurd function](#). But this doesn't quite work out in C++ because of calling convention reasons. Nevertheless, a function pointer that is a constant expression of type `auto(*) (Args...) -> noreturn_t` can be converted to an `auto(*) (Args...) -> U`. As if this conversion were `constexpr`.

This makes sense from a type-theoretic perspective and makes a lot of other uses just work.

We would then change the type of a *throw-expression* to be `noreturn_t` (instead of `void`). This change, coupled with the conversion rule above, means we'd no longer need the special case in conditional expressions. Consider:

```
condition ? 42 : throw std::runtime_error("oops")
```

The types of the second and third operand are `int` and `noreturn_t`, respectively. `noreturn_t` is convertible to `int` (because it is convertible to anything), but `int` is not convertible to `noreturn_t` (since nothing is), thus the result of the expression has type `int`, as desired.

We would also want to change the signatures of all of our never-returning functions to have this in their types:

```
namespace std {  
- [[noreturn]] auto terminate() -> void;  
+ auto terminate() -> noreturn_t;  
}
```

Which would likewise allow for the desired conditional with `std::terminate()` to work for the same reasons as the conditional with `throw`:

```
condition ? 42 : std::terminate()
```

We're approaching `std::noreturn_t` from the perspective of wanting to detect diverging expressions and statements. But there are other reasons to want to have a bottom type in the type system. Consider...

§ 2.2.1 `std::function<std::noreturn_t()>`

How do you have a callback that signals that it must not terminate? Well, you can't really. Because that's not something you can signal in the type system today. But with a bottom type it can be, and the type `std::function<std::noreturn_t()>` becomes meaningful.

§ 2.2.2 `std::expected<T, std::noreturn_t>`

One is its use in sum types. Consider the type `std::expected<T, std::noreturn_t>`. What are its properties? Well, in general, a `std::expected<T, E>` is holding either a `T` in its valid state or an `E` in its error state, and so its storage is something like a `bool` and a `union { T; E; }`. But if `E` is `std::noreturn_t`, then we know that there *cannot be* an error state. You cannot form a value of type `std::noreturn_t`, so we don't even need to store it at all. The layout of `std::expected<T, std::noreturn_t>` can simply be `struct { T; }`. Likewise for `std::expected<std::noreturn_t, E>`, which can only ever be in the error state.

This is a useful thing to be able to express in the type system, since you might have an API whose contract is that it returns some kind of `expected<T, E>` — but this particular implementation of that API might simply never fail (or always fail). So we can return an `expected<T, noreturn_t>` to conform to the expected shape of the return type while also having all the state checks trivially optimize away (since its `operator bool() const` is trivially just `return true;`). Moreover, a lot of code that expects a particular `E` will continue to work fine since `noreturn_t` is convertible to `E`.

§ 2.2.3 Valueless Ranges

By some coincidence, Barry just ran into this issue (again) while we were writing the initial revision of this paper.

A Range (in the C++20 sense) has a few associated types. Its reference is just `decltype(*it)`, unfortunately named in retrospect since it need not be a reference type. Then, a range has a `value_type` — which is the type you would use if you wanted an independent value. And the range's `value_type` and reference have to be related somehow, they need to have a `common_reference`. For most ranges, there isn't much to think about here — reference is either `T&` or `T const&` for some object type `T`, `value_type` is `T`, everything just falls out straightforwardly.

But there are few situations where this just doesn't work out:

1. Let's say we have an abstract base class, `Abstract`. We can produce a range whose reference is `Abstract&` (let's say we don't want to deal with pointers, since we know none of ours are ever null). But we simply *cannot* produce a `value_type` of `Abstract`. It's... abstract. There are situations where you can get away with it, but in a lot of ranges code, you just can't. This is [\[LWG3864\]](#).
2. Let's say we have a type `S` with a trailing flexible array member. We can produce a range whose reference is `S&`. Unlike `Abstract`, it might be valid to form an object of type `S` — it would just be semantically wrong to do so. Moreover, enumerating such a range would end up producing a `value_type` of `tuple<ptrdiff_t, S>`, which in `libstdc++`'s layout is `struct { S; ptrdiff_t; }`. That puts the flexible array member in the middle of the struct, and gcc 14 starts rejecting this code.
3. Similar to (2), there are other cases of dynamically sized ranges. For instance, rather than a `span<T>` I might want to produce an `erased_span<Base>` which is a range of `Base&` but whose dynamic type (and size) is chosen at runtime. Any code that attempts to actually produce a `value_type` of `Base` would be logically wrong — even if it might compile.

In all of these cases, what we really want to be able to say is: there *is* no `value_type`. Any algorithm that attempts to produce one is broken.

A bottom type solves this quite nicely. Adding `using value_type = std::noreturn_t;` meets the other requirements, as long as `std::noreturn_t&` is convertible to reference. We get a valid C++20 range and the compile errors we get are actual logic errors — using algorithms that try to actually form a `value_type`.

This, in of itself, isn't a complete solution, since all the invocation concepts in Ranges (24.3.6 [\[indirectcallable\]](#)) still will attempt to invoke the provided callable with `value_type&` (or the projected `value_type&`), and we'd need to do something there too. But simply having a `std::noreturn_t` to use here does solve a lot of the problem.

§ 2.3 Doing Our Due Diligence to Deduce `do` Divergence

Consider the following pattern arms:

```

pattern1 => do { std::println("never gonna give you up"); throw 2; };
pattern2 => do { std::println("never gonna let you down");
std::terminate(); };
pattern3 => do { std::println("never gonna run around"); do_return
std::terminate(); };
pattern4 => do { std::println("and desert you"); continue; };
pattern5 => break;

```

The last case here is simple — we can special case escaping statements, and the pattern matching paper already does this.

The first four cases though also all unconditionally diverge. We know that from simply examining the code. But what is the specific language rule by which we could ensure that the pattern match expression can exclude these arms when deducing the type? In *pattern₃*, we actually have a `do_return` statement, so the `do` expression there could straightforwardly be said to have type `noreturn_t` and thus diverge. But the other three have no `do_return` and thus would, by default, end up having type `void` — which we do not want. So what do we do?

One option is to copy the logic from some other languages (like Rust) and also the initial GCC statement-expression model and have an implicit value from `do` expressions. That is, the other three patterns above implicitly take their values from `throw 2`, `std::terminate()`, and `continue` respectively — which all have type `noreturn_t`. So this just works. But the reason we didn't initially want to do this for `do` expressions was that we don't already have a way to turn `if` statements and `for/while` statements into expressions — which makes early returns challenging. From the gcc doc example:

gcc	Proposed
<pre> ({ int y = foo(); int z; if (y > 0) z = y; else z = -y; z; }) </pre>	<pre> do { int y = foo(); if (y > 0) { do_return y; } else { do_return -y; } } </pre>

With implicit last value (or, furthermore, with an `if` expression which operates under implicit last value rules):

Explicit <code>do_return</code>	Implicit last value	An <code>if</code> expression
<pre>do { int y = foo(); if (y > 0) { do_return y; } else { do_return -y; } }</pre>	<pre>do { int y = foo(); if (y > 0) { do_return y; } -y; }</pre>	<pre>do { int y = foo(); if (y > 0) { y; } else { -y; } }</pre>

But we're concerned that the mixing and matching of explicit and implicit yields would be confusing. It's easy to miss the implicit `do_return` in the presence of explicit ones. Likewise, attempting to turn `if` into an expression this late in C++'s lifetime might be too novel? We would also then have to find a way to turn loops into expressions (or resurface `do_return`, again running into the explicit/implicit issue).

So if we don't want to use implicit last value, how else can we deduce divergence? We suggest the following rule, inspired by implicit last value:

- 1 An expression is a *diverging expression* if its type is `noreturn_t`.
- 2 A *statement* is a *diverging statement* if it is:
 - (2.1) — A *compound-statement* where the last statement is a diverging statement,
 - (2.2) — an *escape-statement*, See [\[P2688R4\]](#)
 - (2.3) — a *statement-expression* whose *expression* is a diverging expression, or
 - (2.4) — an `if` statement with an `else` branch, where both substatements are diverging statements.
 - (2.5) — a `constexpr if` statement where the taken substatement is a diverging statement.
- 3 The type of a *do-expression* is determined as follows.
 - (3.1) — If there is a *trailing-return-type* that is not a placeholder, that type.
 - (3.2) — Otherwise, if there are any non-discard `do_return` statements within the body of the *do-expression*, let T be the type deduced from them. If the type deduced is not the same in each deduction, the program is ill-formed. Otherwise, the type is T.
 - (3.3) — Otherwise, if the last *statement* is a diverging statement, then `noreturn_t`.
 - (3.4) — Otherwise, `void`.

If there are any `do_return` statements, then the `do` expression doesn't diverge. It might produce a value. Maybe that `do_return` statement is logically unreachable, but we can't in general determine that. But if there are no `do_return` statements, we need to see if we actually diverge. Which is non-trivial because we need to handle things like:

```

// just an escaping-statement
do { continue; }

// a statement-expression that is a diverging expression
do { std::terminate(); }

// an if statement with just an if, this is NOT diverging
// because if the condition is false, then we're just a void expression
do { if (condition) { throw 1; } }

// but this one DOES diverge (just differently in both branches)
do { if (condition) { continue; } else { break; } }

// this diverges if the condition is true
do { if constexpr (condition) { std::terminate(); } }

```

This rule ensures that all of our initial pattern arms diverge, as desired:

```

pattern1 => do { std::println("never gonna make you cry"); throw 2; };
pattern2  => do { std::println("never gonna say goodbye");
               std::terminate(); };
pattern3 => do { std::println("never gonna tell a lie"); do_return
               std::terminate(); };
pattern4 => do { std::println("and hurt you"); continue; };
pattern5 => break;

```

§ 2.4 Alternative with **[[noreturn]]**

The advantage of the approach described with `noreturn_t` is that diverging expressions can appear in the type system. This provides a better solution than **[[noreturn]]**, in a way that can further generalize to other scenarios (as we showed earlier). We can have a `std::function<std::noreturn_t()>`, we can have `std::expected<T, std::noreturn_t>`, valueless ranges, etc.

But an alternative approach would be to avoid changing any of the existing functions (like `std::abort()`, `std::terminate()`, etc.) or changing the type of a *throw-expression*, and instead recognize **[[noreturn]]** more explicitly.

The rule we lay out above would instead become:

1 A *statement* is a *diverging statement* if it is:

- (1.1) — A *compound-statement* where the last statement is a diverging statement,
- (1.2) — an *escaping-statement*,
- (1.3) — a *statement-expression* whose *expression* is a diverging expression, or

- (1.4) — an `if` statement with an `else` branch, where both substatements are diverging statements.
- (1.5) — a `constexpr if` statement where the taken substatement is a diverging statement.
- 2 An expression is a *diverging expression* if:
 - (2.1) — it is an invocation of a `[[noreturn]]` function,
 - (2.2) — it is a *throw-expression*, or
 - (2.3) — it is a *do-expression* whose type is `void` and whose last statement is a diverging statement.

This is a simpler change. It doesn't involve introducing a new language/library type or changing existing standard library functions or the meanings of some code — although we doubt too many people are relying on `decltype(throw 1)` being `void`. However, it doesn't compose. `[[noreturn]]` isn't deduced, so wrapping becomes challenging.

Leaning on `[[noreturn]]` also means that we end up leaning on `void` even harder to mean two completely different things: `void f() { }` is a function that returns a value, while `[[noreturn]] void g() { std::exit(-1); }` is a function that doesn't.

§ 3 Proposal

We propose to:

- to introduce a new type `std::noreturn_t` which represents an expression with no value, that unconditionally diverges.
 - `std::noreturn_t` is convertible to any type (include reference types)
 - `std::noreturn_t` is not constructible, and you cannot `reinterpret_cast` into it, `std::bit_cast` into it, `static_cast` into it, etc.
 - `std::noreturn_t(*) (Args...)` is convertible to `R(*) (Args...)` for all `R` (again, including reference types), but only if the source function pointer is a constant expression (i.e. this conversion behaves as if it is a `constexpr` function).
- change all the standard library functions that are currently `[[noreturn]] void f()` to instead be `std::noreturn_t f()`
- change the type of *throw-expression* to `std::noreturn_t` and remove the current... uh... exception for exceptions in the conditional operator (it will be subsumed by the usual convertibility rule).
- adopt the same rules for `do` expressions [\[P2806R2\]](#).

§ 4 References

[LWG3864] Barry Revzin. zip over range of reference to an abstract type.

<https://wg21.link/lwg3864>

[P2688R4] Michael Park. 2024-12-17. Pattern Matching: match Expression.

<https://wg21.link/p1771r1>

[P2806R2] Barry Revzin, Bruno Cardoso Lopez, Zach Laine, Michael Park. 2023-11-16. do expressions.

<https://wg21.link/p2806r2>